PROG6212 POE PART 1

Malose Kamogelo Matjiu

ST10474956

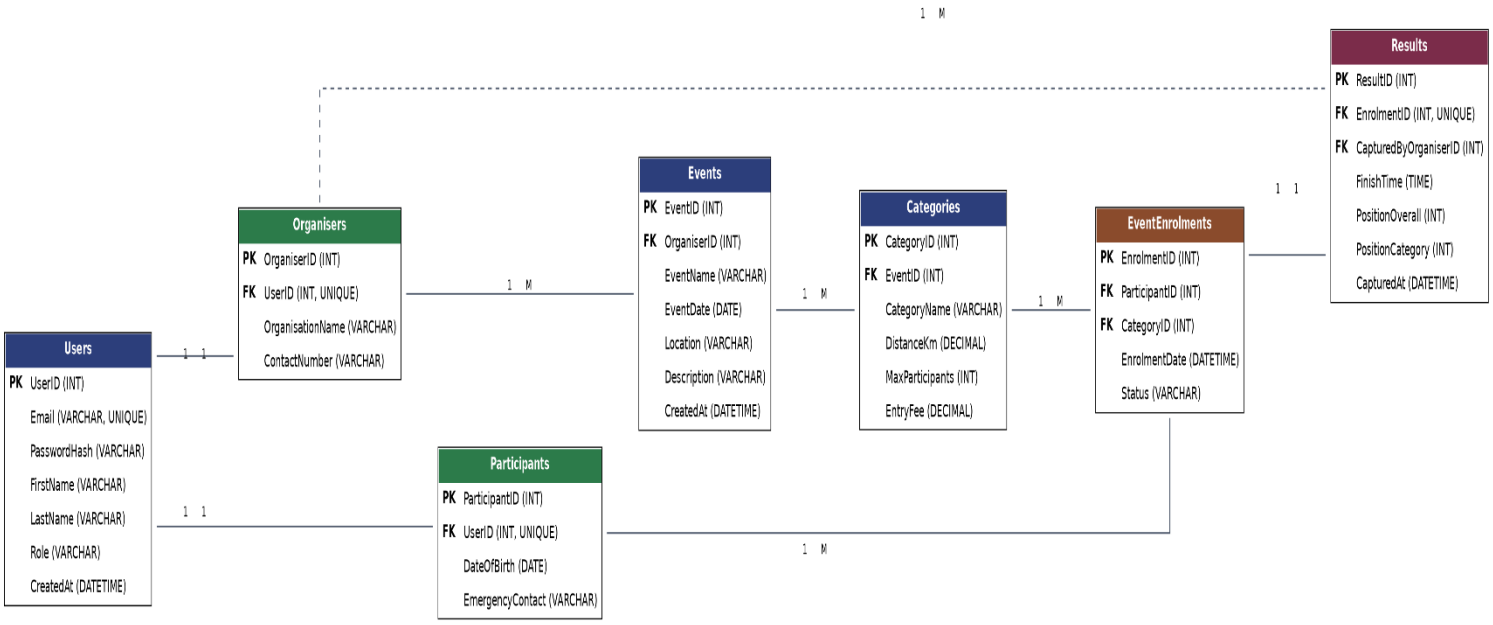
Part 1, Section A

Users

```
│
├── 1:M ── Events
│
└── 1:M ── Enrolments
        │
        ├── M:1 ── Events
        ├── M:1 ── Categories
        └── 1:0..1 ── Results
```

Events

```
│
├── 1:M ── Categories
├── 1:M ── Enrolments
├── 1:0..M ── Weather
└── 1:0..1 ── Routes
```



RaceDay API Endpoint Plan

Part 1, Section B

This table lists every endpoint the RaceDay REST API will expose in Part 2.

It covers Authentication, User Profile, Events, Categories, Event Enrolments,

and Results, as required. All routes are prefixed with `/api/`.

1. Authentication

HTTP Method, Route, Description, Role Required, Request Body, Expected Response,

(POST)/api/auth/register. Creates a new user account as either an Organiser or a Participant. None (public) | { firstName, lastName, email, password, role } 201 Created – user created; 400 Bad Request – validation error; 409 Conflict – email already registered

| POST | /api/auth/login | Authenticates a user and returns a JWT access token. None (public) | { email, password } 200 OK – token + role returned; 401 Unauthorized – invalid credentials

2. User Profile

| HTTP Method | Route | Description | Role Required | Request Body | Expected Response

| GET | /api/users/me | Returns the profile of the currently logged-in user. Any (logged in) | None | 200 OK – user profile object; 401 Unauthorized .

| PUT | /api/users/me Updates the profile details of the logged-in user. Any (logged in) { firstName, lastName, contactNumber } 200 OK – updated profile; 400 Bad Request

| DELETE | /api/users/me Deletes the logged-in user's account. Any (logged in) None 204 No Content; 404 Not Found

3. Events

| HTTP Method | Route | Description | Role Required | Request Body | Expected Response

| GET | /api/events | Lists all upcoming events. Supports optional filtering by location/date. None (public) None 200 OK – array of events

| GET | /api/events/{id} Returns the full detail of a single event, including its categories. None (public) None 200 OK – event object; 404 Not Found

| POST | /api/events Creates a new event. Organiser { eventName, eventDate, location, description } 201 Created – new event; 400 Bad Request

| PUT | /api/events/{id} | Updates an existing event owned by the logged-in Organiser. | Organiser | `{ eventName, eventDate, location, description }` | 200 OK – updated event; 403 Forbidden – not the owner; 404 Not Found |

| DELETE | /api/events/{id} Deletes an event owned by the logged-in Organiser. Organiser None 204 No Content; 403 Forbidden; 404 Not Found

4. Categories

| HTTP Method | Route | Description | Role Required | Request Body | Expected Response |

| GET | /api/events/{eventId}/categories | Lists all categories for a specific event. | None (public) | None | 200 OK – array of categories; 404 Not Found |

| POST | /api/events/{eventId}/categories | Adds a new category to an event. | Organiser | `{ categoryName, distanceKm, maxParticipants, entryFee }` | 201 Created – new category; 403 Forbidden; 404 Not Found |

| PUT | /api/categories/{id} | Updates an existing category. | Organiser | `{ categoryName, distanceKm, maxParticipants, entryFee }` | 200 OK – updated category; 403 Forbidden; 404 Not Found |

| DELETE | /api/categories/{id} | Removes a category from an event. | Organiser | None | 204 No Content; 403 Forbidden; 404 Not Found |

5. Event Enrolment

| HTTP Method | Route | Description | Role Required | Request Body | Expected Response

| POST | /api/categories/{categoryId}/enrol Enrols the logged-in Participant into a category.Participant { } (participant identified from token) 201 Created – enrolment record; 404 Not Found – category does not exist; 409 Conflict – already enrolled or category full

| GET | /api/enrolments/me Lists all enrolments belonging to the logged-in Participant. Participant None 200 OK – array of enrolments

| GET | /api/events/{eventId}/enrolments Lists all enrolments for an event (for the Organiser managing it). Organiser None 200 OK – array of enrolments; 403 Forbidden; 404 Not Found

| DELETE | /api/enrolments/{id} Cancels an enrolment. Participant None 204 No Content; 403 Forbidden – not the owner; 404 Not Found

6. Results

| HTTP Method | Route | Description | Role Required | Request Body | Expected Response |

| POST | /api/enrolments/{enrolmentId}/results Captures a finish result for a Participant's enrolment. Organiser { finishTime, positionOverall, positionCategory } 201 Created – result record; 403 Forbidden; 404 Not Found; 409 Conflict – result already captured

| GET | /api/results/me | Returns the logged-in Participant's personal result history. Participant None 200 OK – array of results

| GET | /api/events/{eventId}/results Returns all results for an event. None (public) None 200 OK – array of results; 404 Not Found

| PUT | /api/results/{id} Corrects a previously captured result. Organiser { finishTime, positionOverall, positionCategory } 200 OK – updated result; 403 Forbidden; 404 Not Found

Design notes

-**Role enforcement:** happens via the JWT claims issued at login — the `role` claim is checked in an `[Authorize(Roles = "Organiser")]` (or equivalent) attribute on every protected endpoint in Part 2.

- Routes that modify a resource (`PUT`/`DELETE`) always confirm the logged-in user actually **owns** that resource (e.g. an Organiser can only edit their own events) before allowing the change — this is why several endpoints list a `403 Forbidden` response in addition to `404 Not Found`.

- This plan matches the ERD in `RaceDay_ERD.png` exactly: one endpoint group per entity, plus the `EventEnrolments` join endpoints that resolve the many-to-many relationship between Participants and Categories.

Validate Part 1 Documentation

```
# Runs on every push/PR to main so the marker can see a green check mark
# in the README before the submission deadline.
on:
  push:
    branches: [ main ]
  pull_request:
    branches: [ main ]
  workflow_dispatch:

jobs:
  validate-docs-folder:
    runs-on: ubuntu-latest
    steps:
      - name: Check out repository
        uses: actions/checkout@v4

      - name: Check /docs folder exists
        run: |
          if [ ! -d "docs" ]; then
            echo "::error::The /docs folder is missing."
            exit 1
          fi
          echo "/docs folder found."

      - name: Check required Part 1 files exist
        run: |
```

```
        missing=0

        if ! ls docs/*.png >/dev/null 2>&1 && ! ls docs/*.pdf >/dev/null 2>&1; then
          echo "::error::No ERD file (.png or .pdf) found in /docs."
          missing=1
        fi

        if ! ls docs/*Endpoint*.md >/dev/null 2>&1 && ! ls docs/*Endpoint*.pdf >/dev/null
2>&1; then
          echo "::error::No endpoint plan file found in /docs."
          missing=1
        fi

        if ! ls docs/*.sql >/dev/null 2>&1; then
          echo "::error::No .sql schema file found in /docs."
          missing=1
        fi

        if [ "$missing" -eq 1 ]; then
          exit 1
        fi

        echo "All required Part 1 documents are present."

      - name: Check README exists
        run: |
          if [ ! -f "README.md" ]; then
            echo "::error::README.md is missing from the repository root."
```

```
  exit 1
fi
echo "README.md found."
```

Part 1, Section C

```
-- RaceDay Database Schema

CREATE DATABASE RaceDayDB;

USE RaceDayDB;

-- Table: Roles
CREATE TABLE Roles (
   RoleID INT IDENTITY(1,1) PRIMARY KEY,
   RoleName VARCHAR(20) NOT NULL UNIQUE
);

-- Table: Users
CREATE TABLE Users (
   UserID INT IDENTITY(1,1) PRIMARY KEY,
   FullName VARCHAR(100) NOT NULL,
   Email VARCHAR(150) NOT NULL UNIQUE,
   PasswordHash VARCHAR(255) NOT NULL,
   RoleID INT NOT NULL,
   ProfilePictureURL VARCHAR(500) NULL,
   CreatedAt DATETIME NOT NULL DEFAULT GETDATE(),
   CONSTRAINT FK_Users_Roles FOREIGN KEY (RoleID) REFERENCES Roles(RoleID)
```

```
);

-- Table: Events
-- Created and owned by an Organiser (Users)
CREATE TABLE Events (
    EventID INT IDENTITY(1,1) PRIMARY KEY,
    Name VARCHAR(150) NOT NULL,
    Description VARCHAR(500) NULL,
    EventDate DATETIME NOT NULL,
    Location VARCHAR(150) NOT NULL,
    DistanceKm DECIMAL(5,2) NOT NULL,
    EventType VARCHAR(10) NOT NULL,
    BannerImageURL VARCHAR(500) NULL,
    OrganiserID INT NOT NULL,
    CreatedAt DATETIME NOT NULL DEFAULT GETDATE(),
    CONSTRAINT FK_Events_Users FOREIGN KEY (OrganiserID) REFERENCES
Users(UserID),
    CONSTRAINT CK_Events_EventType CHECK (EventType IN ('Run', 'Walk', 'Cycle'))
);

-- Table: Categories
CREATE TABLE Categories (
    CategoryID INT IDENTITY(1,1) PRIMARY KEY,
    EventID INT NOT NULL,
    Name VARCHAR(100) NOT NULL,
    DistanceKm DECIMAL(5,2) NULL, -- NULL for age-based categories ('Under 20',
'Senior')
    MaxParticipants INT NOT NULL DEFAULT 100,
```

```
   CONSTRAINT FK_Categories_Events FOREIGN KEY (EventID) REFERENCES
Events(EventID)

);


-- Table: Enrolments

CREATE TABLE Enrolments (

   EnrolmentID INT IDENTITY(1,1) PRIMARY KEY,

   UserID INT NOT NULL,

   CategoryID INT NOT NULL,

   EnrolmentDate DATETIME NOT NULL DEFAULT GETDATE(),

   Status VARCHAR(20) NOT NULL DEFAULT 'Confirmed',

   CONSTRAINT FK_Enrolments_Users FOREIGN KEY (UserID) REFERENCES
dbo.Users(UserID),

   CONSTRAINT FK_Enrolments_Categories FOREIGN KEY (CategoryID) REFERENCES
Categories(CategoryID),

   CONSTRAINT UQ_Enrolments_User_Category UNIQUE (UserID, CategoryID)

);


-- Table: Results

CREATE TABLE Results (

   ResultID INT IDENTITY(1,1) PRIMARY KEY,

   EnrolmentID INT NOT NULL UNIQUE,

   FinishTime TIME NULL,

   Position INT NULL,

   Status VARCHAR(20) NOT NULL DEFAULT 'Finished',

   CONSTRAINT FK_Results_Enrolments FOREIGN KEY (EnrolmentID) REFERENCES
Enrolments(EnrolmentID)

);
```

```
-- Roles
INSERT INTO Roles (RoleName) VALUES ('Organiser'), ('Participant');

-- Users: 2 Organisers, 3 Participants
INSERT INTO Users (FullName, Email, PasswordHash, RoleID, ProfilePictureURL)
VALUES
  ('Thabo Nkosi', 'thabo.nkosi@raceday.co.za', 'hashed_pw_1', 1, NULL), -- Organiser
  ('Lindiwe Dube', 'lindiwe.dube@raceday.co.za', 'hashed_pw_2', 1, NULL), -- Organiser
  ('Sipho Mokoena', 'sipho.mokoena@example.com', 'hash_pw_3', 2, NULL), -- Participant
  ('Anika van Wyk', 'anika.vanwyk@example.com', 'hash_pw_4', 2, NULL), -- Participant
  ('Naledi Molefe', 'naledi.molefe@example.com', 'hash_pw_5', 2, NULL); -- Participant

-- Events:
INSERT INTO Events (Name, Description, EventDate, Location, DistanceKm, EventType, BannerImageURL, OrganiserID)
VALUES
  ('Johannesburg City Run', 'Annual road running event through the Johannesburg CBD.', '2026-11-14 07:00:00', 'Johannesburg, Gauteng', 10.0, 'Run', NULL, 1),
  ('Cape Town Cycle Tour', 'Scenic cycling event around the Cape Peninsula.', '2026-03-08 06:30:00', 'Cape Town, Western Cape', 109.0, 'Cycle', NULL, 2),
  ('Durban Beachfront Walk', 'Family-friendly charity walk along the Durban beachfront.', '2026-09-20 08:00:00', 'Durban, KwaZulu-Natal', 3.0, 'Walk', NULL, 1);

-- Categories:
INSERT INTO Categories (EventID, Name, DistanceKm, MaxParticipants)
VALUES
  (1, '5km Fun Run', 5.0, 500),
  (1, '10km Road Race',10.0, 300),
```

```
   (2, '109km Cycle', 109.0,1000),
   (2, '55km Cycle', 55.0, 800),
   (3, '3km Walk', 3.0, 400),
   (3, 'Under 20', NULL, 150);

-- Enrolments: participants enrolled in categories
INSERT INTO Enrolments (UserID, CategoryID, Status)
VALUES
   (3, 1, 'Confirmed'), -- Sipho -> 5km Fun Run (Joburg)
   (4, 2, 'Confirmed'), -- Anika -> 10km Road Race (Joburg)
   (5, 3, 'Confirmed'), -- Naledi -> 109km Cycle (Cape Town)
   (3, 5, 'Confirmed'); -- Sipho -> 3km Walk (Durban)

-- Results: results for two of the enrolments
INSERT INTO Results (EnrolmentID, FinishTime, Position, Status)
VALUES
   (1, '00:24:12', 5, 'Finished'),
   (2, '00:52:47', 12, 'Finished');

PRINT 'RaceDay database schema created and seeded successfully.';

SELECT * FROM Roles;
SELECT * FROM Users;
SELECT * FROM Events;
SELECT * FROM Categories;
SELECT * FROM Enrolments;
SELECT * FROM Results;
```

Results | Messages

RoleID	RoleName

UserID	FullName	Email	PasswordHash	RoleID	ProfilePictureURL	CreatedAt

EventID	Name	Description	EventDate	Location	DistanceKm	EventType	BannerImageURL	OrganiserID	CreatedAt

CategoryID	EventID	Name	DistanceKm	MaxParticipants

EnrolmentID	UserID	CategoryID	EnrolmentDate	Status

ResultID	EnrolmentID	FinishTime	Position	Status